

When does reusing a solver grid pay for an exact point-in-box particle count?

Anton Björkegren

22 September 2026, revision 3

A controlled comparative study with a measured negative answer.

Abstract

A keep-out decision asks whether any material intrudes into a declared axis-aligned box. A natural implementation reuses the simulator’s own mass grid as a broad phase: reject a box if every grid node within a pad has zero mass, otherwise run the exact particle sweep. The bounded question of this study is **not** whether that prefilter is exact — it is — but *when it pays* relative to a competent exact accelerator that answers the same question from the same particles. We compare, on frozen states and identical output semantics (an integer strict particle-centre count per box), three exact paths: (i) the original implementation’s grid prefilter with an exact fallback, (ii) a uniform cell list built by histogram/prefix-scan/scatter, and (iii) a radix-sorted cell key with binary search. All paths agree with two independent oracles in every tested case. On complete per-frame cost, the best exact accelerator is **0.20-0.74 of the grid path** (the grid is 1.4–4.9× slower), and it also wins in the query-only regime with all capture, binding and validation removed. The original implementation’s apparent gain of 0.647 against the *direct* exact kernel is real but is erased as soon as the comparison includes a spatial index built from the particles. The grid path does beat the radix-sorted-key index on several cells (for example 42.70 ms against 70.93 ms on sparse 1024), so the decisive comparison is against the *best* of the two indexes, which is the cell list on every tested cell. We report the mechanisms that are measured: the pad-inflated gate still forwards a large fraction of boxes (16.5 % sparse, 41–50 % dense/corner, 70 % thin slab), and the complete grid path carries conversion and ownership costs (copy, P2G, bind re-verification, validation, occupancy) that a particle-derived index does not. The remaining auditable content of the line is exactness and a set of negative controls, not a speedup.

1. Introduction and the bounded question

A keep-out volume is declared once but judged against material that arrives: a pile that spreads, a stream that falls, a bin that fills. When the occupied set itself moves, a static abstraction of the particles (a convex hull or any fixed body fitted to them) contains the particles and more, so it over-reports the occupied set. A test that asks whether any material intrudes then becomes a test against a set that is not the material. A dynamic occupancy derived from the simulator’s own state is the natural alternative feed.

That alternative is attractive because the simulator already deposits particles onto a mass grid; in principle the grid can serve as a cheap conservative prefilter for the exact question “how many particle centres lie in this box?”. The prefilter is conservative in the right direction: it may forward a box needlessly, but under stated assumptions it must not reject a box that contains a particle. Earlier work on this line measured the prefilter’s complete cost against the *direct* exact particle sweep and reported a gain on sparse, many-box cells. The open question is whether that gain survives a **matched** comparison — that is, a comparison against an exact accelerator that is itself built from the same particles and answers the same question exactly, including its own build/update cost. This is the question investigated here:

On the same captured solver state and the same strict particle-in-box count, does reuse of the solver’s mass grid beat a competent exact spatial accelerator, including the accelerator’s build/update cost?

The measured answer is **no** against the best exact accelerator, on every tested cell; against the unindexed direct sweep it is a qualified **yes** (the grid wins on five of the eight measured cells, by a smaller margin). Two consequences follow and are developed in this study. First, the quantity that must be reported for such a prefilter is the explicit per-run ratio of the grid path to the *best exact accelerator*, not a ratio to the unindexed direct sweep. Second, the mechanisms that decide the outcome are the gate's filter inflation (how many boxes it still forwards) and candidate density (how much work each forwarded box costs), together with the conversion and ownership costs the grid path must pay before any query.

For brevity we call the grid-prefilter implementation under study the *original implementation* and the separately reproduced measurement harness the *independent reproduction*; the source labels in the tables use these two scientific roles.

This study is deliberately a comparative empirical study. It does not claim a new algorithm: the cell list is a textbook broad phase and the engine already ships a fixed-radius hash-grid broad phase. It also does not claim that the original implementation's prefilter is wrong; on the audited scope it is exact, and that exactness is independently reproduced.

2. The exact count and the contracts that must not be conflated

The subject is one integer question. Let x_i be particle centres, dx the grid pitch, r a fixed radius, and $[lo, hi]$ the declared box.

- **D1 (strict).** A breach exists iff a particle **centre** lies in the closed box $[lo, hi]$. This is the machine fact and the reference count of this study.
- **D2 (dilated-box count).** A breach exists iff a particle centre lies within the Euclidean distance r of the box, i.e. if the box dilated by r in the Euclidean (rounded-box) sense contains a particle centre. This is a rounded-box dilation, not an axiswise L-infinity expansion: the Euclidean r -neighbourhood of an axis-aligned box rounds its edges and corners. D2 is a finite-radius particle/box intersection question; it is not an intrinsically raster quantity.
- **D3 (cell occupancy).** A breach exists iff a node whose centre is in the box carries mass above a threshold, $mass > \theta * \rho * dx^3$.

D1, D2 and D3 are different questions. A feed that answers one answers the others incorrectly unless a conservative relation is proved. The prefilter is sound only as a *rejection* test for D1/D2: it may over-flag, but it must never silently drop a breach. False positives (over-flagging) and false negatives (missed breaches) are always reported separately and always relative to a named reference.

Four contracts are kept separate throughout and are not summed: dynamic signed distance (out of scope; reported only as a negative), occupancy/owner-indexed occupancy (used as the feed), particle keep-out count (the subject), and body/region/material semantics (out of scope). A boolean occupancy channel loses material share and part identity and must not stand in for region semantics.

The three compared paths all return exactly D1 as an integer per box. They differ only in how the candidate particles are found.

3. Paths under comparison

Grid prefilter with exact fallback (original implementation). For each box, the path tests the occupancy of the captured mass grid over the box inflated axis-wise by $pad = r + 2 * dx + D$, where D is the maximum particle displacement since the grid was bound. If every node within the pad has zero mass the box is rejected with count 0; otherwise the box is flagged and an exact particle sweep is run for it. The grid is usable only when it is bound to the particle state: the implementation either builds the grid as its own fixed-point P2G(x_{at} , m_{at}) or verifies a supplied grid bit-for-bit against a fresh P2G, and re-verifies the binding at bind time. A stale grid (particles newer than the grid) or a future grid (grid newer than the particles) falls back to the exact sweep

on the state's own positions. The pad is sound under the quadratic B-spline deposition assumption: the nearest node to a particle is at most $0.5 \cdot dx$ away per axis and carries positive mass, so a particle in the box implies a positive node within pad.

Exact uniform cell list (baseline A). A grid broad phase built from the particles at the same pitch: one integer cell index per particle, an atomic histogram over res^3 cells, an exclusive prefix scan, and an atomic scatter into a compact particle order. A query walks the clamped cell range overlapping the box and applies the exact D1 predicate only to the particles in those cells. This is the same “grid broad phase + exact narrow phase” pattern as the engine's own fixed-radius hash-grid broad phase, adapted to an axis-aligned box range.

Exact radix-sorted cell key (baseline B). A second exact accelerator: one `int64` cell key per particle, then a two-pass radix sort of (key, index). A query binary-searches the sorted key array for the z-interval of each (ix, iy) row and applies the exact D1 predicate over that key range. No prefix scan is used.

A **direct** exact sweep over all particles per box is carried as an unindexed reference, because it is the denominator against which the original-implementation gain was originally reported. It is not a matched baseline: it has no build cost and no candidate restriction.

4. Measurement protocol

Frozen states. Five states are frozen from the simulator checkpoint and not regenerated during measurement: a family around the intrusion window plus a shifted-geometry variant with a baffle displaced by -20 mm in z. They hold 121 000–161 000 particles and a 64^3 resident mass grid ($dx = 0.0125$ m, $\rho = 1600$ kg/m³, origin (0, -0.40 , -0.05)). A separate reproduction assembled two **new** states from the frozen ones by a deterministic translation plus a fixed permutation with fresh identity labels; no simulator was run. Full state hashes are in the package.

Box families and seeds. Boxes are generated from five frozen geometry families: *sparse* (small boxes over the whole domain), *allnear* (boxes restricted to the material region), *slab* (thin in z, wide in x/y), *corner* (clustered at the lower domain corner), and *empty* (a region with no particles). Box counts are 1, 32, 256 and 1024; query multiplicity Q is 1, 4 or 16. Two different box-seed generators are used by the two measurement sources and are kept distinct here. The matched comparison (Tables 1–2) uses $seed = 81000 + 7000 \cdot ci + 7919 \cdot j$, where ci is the index of the cell in the filtered selection order and j is the query-batch index. The original-implementation complete-cost series (the gain-against-direct numbers in §6) uses a different lock-derived generator, $seed = base_seed + ci$ for the cells it pins and $seed = base_seed + 50000 + ci$ otherwise, with per-query boxes at $seed + 7919 \cdot q$ and $base_seed = 71000$. Because the selection order depends on which cells a run selects, the same cell label can use different boxes in different runs; this is documented rather than hidden, and the raw files that carry a seed field are the original-implementation complete-cost files. The matched, query-sweep and reproduction raw files do **not** store their per-scenario seeds or box coordinates, so the decisive box set is a property record rather than a byte-replay: the package reproduces the displayed *ratios* from the frozen medians, not the individual boxes. The correctness replay uses the stable generator $seed = 90000 + nb + crc32("frame|family") \bmod 1000$, which is deterministic across processes; the frozen original-implementation agreement matrix was generated with a per-process `hash()` and is likewise a property record.

Cost definition and charging. A “complete per-frame” row charges one capture/build for the frame followed by Q query batches: the state copy, the capture (copy plus P2G where the path consumes a grid), the bind (re-verification plus validation plus occupancy where applicable), and Q queries. For the grid path this is the *amortized* form (bind once, then Q queries); a second form re-binds per query. The cell list and sorted key are measured with their build charged once per frame and, separately, in the reusable regime. Every stage is synchronised around the timer. The per-stage medians in §8 are separate per-stage medians measured over the frame; their sum is not the measured complete route (the complete route is timed end to end around the capture, bind and query loop), and the two figures are therefore reported separately rather than combined.

Time boundary and device contention. Recorded total device memory usage on the shared GPU ranged from 2043 to 2353 MiB (approximately 2.00–2.30 GiB) across the bundled measurement records. The carried quantities are therefore the paired 5-repeat medians and the same-run ratios, never the absolute milliseconds in isolation. Every table in the package labels both device and time_boundary.

No cross-run combination. A ratio is computed only from the two medians of one run. Medians from different runs are never combined into a single “measured path”. Where the original-implementation gain against the direct kernel is quoted, the denominator is the direct sweep from the same run.

Correctness before timing. Zero count mismatch is the precondition for any timing statement. The original implementation measured a 99-case held-out batch set on two new frames with independent NumPy and independent Warp D1 references; this study measured 16 cases across six paths; and the independent reproduction measured 10 further cases on the two new states with three independent oracles. All report zero mismatch and zero gate false negatives.

5. Correctness and gate safety (positive, bounded)

All exact paths agree on every tested case. The original implementation’s 99 held-out case-batches (frames 160 and 160-shifted, sparse/allnear/slab/corner/empty, 1/32/256/1024 boxes, Q 1/4/16, seeds 71xxx) give counts == independent NumPy D1 == independent Warp D1 over all boxes, **0 gate false negatives, 0 count mismatch**, with a canonical decision hash identical across two processes. This study’s correctness matrix (16 cases, six paths) reports all_paths_agree = true and zero_count_mismatch = true. The independent reproduction’s new-state matrix (two states $\times$ five box sets, including boxes partly outside the domain, zero-size boxes, boxes whose faces lie exactly on particle coordinates, and the whole domain) reports zero divergence from three independent oracles and exact inclusivity at the boundary. The gate’s own safety was stressed directly: a displacement of $0.90 \cdot dx$ remains gated and exact; $1.00 \cdot dx$ and $1.05 \cdot dx$, a stale grid (lag 2), and a future grid all fall back to the exact sweep on the current positions; a grid built from the wrong frame is rejected at bind. These are measured properties on the tested scope, not a theorem.

The mass-grid threshold alone (D3) is a different and incorrect decision for D1 on the tested thresholds θ in $\{0, 0.05, 0.25, 0.5\}$. At $\theta = 0.5$ it gives 13 false negatives on 20 matched cases and 170 on 300 fuzz scenes, with 0 false positives; lowering the threshold trades false negatives for false positives ($\theta = 0$ gives 15 false positives on the fuzz set). The $\theta = 0.05$ and 0.25 rows were measured on a constructed linear feed, not the simulator kernel, and are not treated as kernel-independent; only $\theta = 0$ and 0.5 are endpoints in that sense. Two structural reasons are measured: on a real 96^3 grid at $\theta = 0.5$, 239 of 8458 particle columns are entirely empty, and a p95 surface error of at most one cell is not a keep-out guarantee. Because the raw threshold already has false negatives at $\theta > 0$, it is not sound as a rejection test either; a rejection test is sound only if it never drops a breach. The prefilter that is used and audited is a different object: it is the **zero/nonzero occupancy of a verified deposition** evaluated over the box inflated by $\text{pad} = r + 2 \cdot dx + D$, with D the maximum displacement since the grid was bound, and it fails safe to the exact sweep whenever the binding, displacement or frame contract is not met. That padded gate has zero false negatives on the tested holdouts (and on the adversarial zero-overlap constructions in which the bare occupancy at $\text{pad} = 0$ misses every breach). Finite tests alone do not certify arbitrary masses; the statement is bounded to the tested scope.

6. Matched complete cost: the grid loses to the best exact accelerator

The original implementation measured complete cost $Q = 16$ on the frozen states, five paired synchronised repeats. The sign convention matters: the reported complete_gain_vs_direct = 1 - amort/direct is positive exactly when the grid path is *faster* than the direct sweep. Against the **direct** exact kernel the grid path wins on five of the eight cells (run a): sparse 1024 new frame amort/direct = 0.3527 (gain 0.6473), sparse 1024 known frame 0.3537 (gain 0.6463), thin slab 0.9447 (gain 0.0553), dense allnear 0.9710 (gain 0.0290), and corner 0.8213 (gain 0.1787). It loses

on three cells: known dense frame amort/direct = 1.0836 (gain -0.0836) and the two small box counts (amort/direct 1.7559 and 1.7537; gains -0.7559 and -0.7537). The gains on slab, dense and corner are therefore smaller *wins*, not losses. The worst complete amort/direct ratio is 1.7559 and the worst full/direct ratio (rebinding per query) is 4.6719. Run b reproduces the same classification with small variation.

Against the **matched exact accelerator** the picture reverses. Table 1 gives the explicit same-run ratio $t_{\text{gate_over_best}} = t_{\text{gate}} / \min(t_{\text{cell_list}}, t_{\text{sorted_key}})$, where a value above 1 means the grid path is slower. The best exact baseline is always the cell list.

Table 1. Original implementation matched complete cost, $q = 16$, same-run ratio (above 1 = grid slower).

Table 1, panel A of 2 — baseline times

state / family (run)	boxes	flagged	t_cell_list (ms)	t_sorted_key (ms)
new frame, sparse (a)	1024	169	16.27	70.93
known frame, sparse (a)	1024	190	16.07	71.28
new geometry, slab (a)	1024	714	89.80	728.32
dense allnear (b)	256	127	7.15	8.15
corner (b)	256	105	7.16	8.25
sparse small (b)	32	32	10.39	36.80
sparse tiny (b)	1	1	6.95	7.54

Table 1, panel B of 2 — same-run ratios

state / family (run)	boxes	t_gate / best	t_gate / direct
new frame, sparse (a)	1024	2.625	0.331
known frame, sparse (a)	1024	2.517	0.340
new geometry, slab (a)	1024	1.353	0.950
dense allnear (b)	256	4.907	0.979
corner (b)	256	4.240	0.841
sparse small (b)	32	1.774	1.823
sparse tiny (b)	1	2.596	1.735

The original implementation's gain is thus conditional in a stronger sense than "geometry-dependent": it is a gain against an unindexed sweep and it disappears against the cell list built from the same particles. The cell list is 0.20–0.74 of the grid path, i.e. the grid is 1.4–4.9× slower. The negative is *not* robust to the baseline choice, however. On several cells the grid path is faster than the radix-sorted-key index, for example $t_{\text{gate}} / t_{\text{sorted_key}} = 42.70/70.93 = 0.60$ on new sparse 1024 and $121.52/728.32 = 0.17$ on the slab; the sorted key is slower than the grid there. The supported statement is therefore that the grid loses to the **best** exact accelerator (the cell list on every tested cell), not that it loses to both accelerators or to every exact baseline.

The q sweep sharpens the point. On the two cells present at every q , the grid loses at $q = 1$ already, when setup dominates and the comparison is least favourable to the accelerator:

Table 2. q sweep, same-run ratios (above 1 = grid slower).

Table 2 — query sweep

cell	Q	t_gate / best	t_gate / direct	gate / cell, query-only
new sparse 1024	1	2.695	1.229	2.113
new sparse 1024	4	2.627	0.549	2.189

Table 2 — query sweep

cell	Q	t_gate / best	t_gate / direct	gate / cell, query-only
new sparse 1024	16	2.625	0.331	2.499
dense allnear 256	1	3.123	2.217	7.154
dense allnear 256	4	3.587	1.481	6.853

The ranking is unchanged when all setup is removed: in the reusable query-only regime the grid path is still 2.1–2.5× slower than the cell list on sparse 1024 and about 7× slower on dense allnear. The negative result is therefore a cost-structure fact, not an artefact of charging capture or validation.

7. Independent reproduction

The independent reproduction re-timed one positive and two negative cells with its own harness and its own charging, on the same frozen states, in two separate processes. Table 3 places the original implementation and the reproduction measurements side by side, each row labelled by source and run; the two sources are never merged into one series.

Table 3. Original implementation and independent reproduction, same-run ratios (above 1 = grid slower).**Table 3, panel A of 2 — same-run times**

cell	source	run	t_gate (ms)	t_best (ms)	t_gate / best
new sparse 1024	original	a	42.70	16.27	2.625
new sparse 1024	reproduction	a	42.83	16.32	2.624
new sparse 1024	reproduction	b	44.09	16.60	2.655
dense allnear 256	reproduction	a	35.07	6.86	5.116
dense allnear 256	reproduction	b	36.39	7.38	4.929
sparse tiny 1	reproduction	a	17.87	7.10	2.518
sparse tiny 1	reproduction	b	19.06	7.08	2.693

Table 3, panel B of 2 — same-run ratios

cell	source	run	t_gate / direct
new sparse 1024	original	a	0.331
new sparse 1024	reproduction	a	0.326
new sparse 1024	reproduction	b	0.329
dense allnear 256	reproduction	a	0.953
dense allnear 256	reproduction	b	0.967
sparse tiny 1	reproduction	a	1.758
sparse tiny 1	reproduction	b	1.939

The independent reproduction reproduces the original implementation’s positive-against-direct and negative-against-index result within run-to-run variation, and its independent query-only ratios agree (2.5–2.6 on sparse 1024, 6.2–7.7 on dense allnear, about 2.1 on the tiny cell). The independent reproduction also identified a defect in the original implementation’s stored summary field `gain_vs_best_exact_baseline`, which carried inconsistent signs and magnitudes across files. This study does not quote that field; every ratio above is recomputed from the same-run medians by `scripts/regenerate_tables.py`.

8. Mechanisms (what is measured, and what is not)

Three measured mechanisms explain the negative answer. They are reported as measurements, not as a general theory.

Filter inflation. The gate forwards a box whenever any node within the pad has positive mass. On the tested states the forwarded fraction is $169/1024 = 16.5\%$ (sparse), $190/1024 = 18.6\%$ (known sparse), $714/1024 = 69.7\%$ (thin slab), $127/256 = 49.6\%$ (dense allnear) and $105/256 = 41.0\%$ (corner). Even on the best sparse cell, one box in six still pays the full exact sweep; on the slab and dense families the gate mostly duplicates the exact work rather than avoiding it.

Candidate density. The cell list tests only the particles that share a cell with the box, and its build already groups particles by cell; the grid path, for a flagged box, scans all particles. On the audited cases the cell-list candidate work per query batch was independently reconstructed and matched an independent NumPy enumeration exactly (`base1_work == independent_work`), so the accelerator is testing exactly the particles a uniform grid must test, with no wasted work. The grid path's flagged boxes therefore do strictly more candidate work than the cell list while also paying for the grid.

Conversion and ownership cost. The complete grid path must copy the state, copy particles and grid, build or verify a P2G, re-verify the binding at bind, validate ids/version/frame/deposition/displacement, and build occupancy. Measured stage medians for the frame are: state/particle copy 0.067 ms, P2G build 0.234 ms, capture from particles 5.067 ms, capture from solver grid 5.668 ms, bind re-verification plus validation 1.745 ms, occupancy 0.051 ms, gated query 2.127 ms, direct exact query 7.127 ms. The particle-derived index pays none of the grid-conversion costs, and its query-only ranking already shows the gap.

What is *not* measured: the occupied material volume, surface clearance, continuous collision, arbitrary (non-axis-aligned) query geometry, production integration, or behaviour on hardware other than the shared test device. The results are for the strict particle-centre count only.

9. Negative controls retained

- 1. The bare mass threshold is an incorrect decision.** Tested thresholds `theta` in `{0, 0.05, 0.25, 0.5}`, contract D3 (`node mass > theta * rho * dx^3`, node centres inside the box). At `theta = 0.5`, 13/20 matched and 170/300 fuzz false negatives, with 0 false positives; at `theta = 0`, 15/300 false positives; at 0.05 and 0.25, 1 and 0 false positives with 22 and 139 false negatives on the fuzz scenes. The 0.05/0.25 rows are on a constructed linear feed rather than the simulator kernel; only 0 and 0.5 are kernel-independent endpoints. We state the tested thresholds and the exact contract rather than a general impossibility: within these tested endpoints no threshold simultaneously removes the false negatives and the false positives, but we do not prove that no threshold on any feed could do so. The audited prefilter is not this bare threshold but the padded zero/nonzero gate of §5, which is exact on the tested scope.
- 2. A particle-fed dynamic occupancy does not fit the tick.** Rebuilding occupancy from 200 000 particles takes a median 84.66 ms against 4.85 ms for a static convex-hull test; an incremental variant takes 35.32 ms and is within 10 ms on 0 of 200 frames; a later exact incremental flow takes 66.52 ms with 3.5 % of frames within 10 ms. The historical 84.66, 4.85 and 66.52 ms times are reported archival values; their original timing runs are not in the package and cannot be recomputed here. The 35.32 ms paired estimate and 3.5 % fraction are computed from the retained records, with the latter from a later reproduction pass.
- 3. The unready public snapshot API produced silent false negatives.** A `dataclasses.replace` with a zero grid and a zero-grid copy kept the verified flag and produced 44 gate false negatives; a cache keyed on a caller-remembered counter produced 1 false negative after an in-place position mutation. The repaired ownership contract closes these paths on the documented public surface (the attacks raise `TypeError / OwnershipError / AttributeError`, and a wrong grid is rejected at bind). **This repaired contract is a prototype:** it depends on an unapplied code patch and `integration_ready = false`; it is not product-integrated code.

4. **An unexpanded gate is not sound for fast motion.** Without a displacement bound the gate misses constructed jumps of two or more cells; with the bound, the measured holdouts give zero false negatives. The displacement bound is required, and the stale/future-grid fallbacks are exact.
5. **An earlier “complete” cost was an accounting artefact.** Timer values in seconds were labelled milliseconds, the gate counter came from the first box batch only, and capture/preparation sat outside the complete loop. The ratios were unit-invariant; the absolute numbers were not. The corrected complete gain is smaller and strictly scoped.

10. Relation to existing work and scope of the contribution

The cell list is a textbook uniform-grid broad phase [7]; the implementation here was written from scratch and no external baseline system was used as the comparison object. The engine’s own fixed-radius hash-grid broad phase is the same “grid broad phase + exact narrow phase” pattern, adapted here to an axis-aligned box range. The work-count equality and the cost reproduction show that this standard accelerator is competent and beats the grid prefilter on every tested cell, including in the reusable regime. The performance contribution claimed for the grid reuse therefore does not exceed reuse of a known broad phase, and this study establishes no methodological novelty. It renders no literature-novelty verdict from missing citations.

The closest published line is incremental signed-distance mapping (FIESTA [1], Voxblox [2], nvblox [3]), which builds and updates a *signed distance* from occupancy/TSDF and outputs distance and gradient for planning. This study asks a different question — a set-membership intrusion count of a simulator’s material state against a declared box — so those systems are not a matched baseline, and a name match is not a comparison. The conservative-query literature (a sharp conservative marching-cubes offset [4], geometric queries on closed implicit surfaces [5], generalized Lipschitz tracing [6]) supplies the kind of statement (a constant conservative offset, guaranteed queries on implicit surfaces) but not a keep-out count, and is not transferred here. What remains auditable in the line is exactness under stated assumptions, a bounded ownership contract, and the negative controls above.

11. Limits and what is not claimed

- **No speedup.** The grid reuse has no surviving performance contribution against a competent exact accelerator. It is presented as a measured negative.
- **Scope.** The grid beats the direct kernel on five of eight measured cells and loses on three; it beats the radix-sorted-key index on several cells (e.g. 42.70 ms against 70.93 ms on sparse 1024). On every tested cell it loses to the cell list, which is the best of the two indexes. The negative result is therefore relative to the cell-list / best baseline, and it persists when setup is removed (query-only), but it is **not** robust to the choice between the two exact indexes.
- **States.** The repair and the matched comparison were audited on a small frozen family and two new state assemblies; the 99 held-out batches are new material, but they are not a population.
- **Ownership.** Only the documented public surface is protected; arbitrary private introspection and raw memory edits are out of scope, and the contract relies on the implementation advancing a version counter. The repair is a prototype, not integration.
- **Feed provenance.** One measured sub-millisecond mass-grid row used a constructed B-spline feed that is not the simulator kernel, and its box lay outside the grid domain; it is excluded from the decision claims.
- **Incremental channel.** The incremental owner channel is exact only under a prefix-removal assumption; a middle removal needs a general set difference. The committed channel is CPU-only.
- **No claim from this study** about occupied material volume, surface clearance, continuous collision, arbitrary query geometry, production integration, or other hardware.

12. Reproduction

A self-contained package accompanies this study: descriptive file names, frozen particle states, the grid prefilter and both exact baselines, the frozen raw measurements, and one script per displayed table. Two commands cover the two different meanings of “reproduce”:

```
# CPU: rebuild every displayed table from frozen raw JSON
python scripts/regenerate_tables.py
# CPU: hashes, state loader, table match, prose controls
python scripts/verify_package.py
```

The tables are recomputed in isolation from the frozen raw records; no printed number is copied into a table source. `python scripts/replay_correctness.py` and `python scripts/replay_benchmark.py` re-run the physical measurements on a GPU (a fresh GPU benchmark is not needed to check the tables), and `python scripts/verify_prose_controls.py` recomputes the supported threshold, mass-grid and dynamic-occupancy controls from retained records in `data/prose_controls/`. The historical 84.66, 4.85 and 66.52 ms timings are reported archival values without original timing runs in the package; they cannot be recomputed here. The 35.32 ms paired estimate and 3.5 % fraction are derived from retained records. The package records full sha256 hashes of every member.

13. References

1. FIESTA: L. Han, F. Gao, B. Zhou, S. Shen, “FIESTA: Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots,” IEEE/RSJ IROS 2019 (arXiv:1903.02144).
2. Voxblox: H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, J. Nieto, “Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning,” IEEE/RSJ IROS 2017 (arXiv:1611.03631).
3. nvblox: A. Millane, H. Oleynikova, E. Wirbel, R. Steiner, V. Ramasamy, D. Tingdahl, R. Siegwart, “nvblox: GPU-Accelerated Incremental Signed Distance Field Mapping,” IEEE ICRA 2024 (arXiv:2311.00626; accepted to ICRA 2024).
4. A. Jacobson, “A Sharp Conservative Offset for Marching Cubes,” arXiv:2609.13430, 2026.
5. T. Huang, “Geometric Queries on Closed Implicit Surfaces for Walk on Stars,” SA Technical Communications '25 (short paper); arXiv:2510.07275, 2025. DOI 10.1145/3757376.3771378.
6. R. Bán, G. Valasek, “Generalized Lipschitz Tracing of Implicit Surfaces,” Computer Graphics Forum, 44(1), e15202, 2025. DOI 10.1111/cgf.15202.
7. C. Ericson, “Real-Time Collision Detection,” Morgan Kaufmann, 2005 (uniform-grid broad phase and spatial hashing); cited for the standard broad phase, not as a matched comparison object.

References [1]–[3] are the closest mapping line and are not used as a matched baseline; [4]–[6] are cited as the conservative-query line. The package’s source map records the exact frozen record behind each displayed number.

Scope note. The results are bounded to the strict particle-centre count on the frozen states and the shared test device; no figure is part of this text.